CSA02 C Programming – CO5 Assignment Answers

Assignment 5: Functions, Recursion, Pointers and Files

Note: These are model answers based on the uploaded assignment sheets. Replace student-specific details and add your own compiler screenshots, timing measurements and GitHub links before submission.

A1 – Decompose blood-bank program

Structure chart: main → load_data, menu/command parser, register_donor, find_eligible, update_stock, save_donors, save_stock, free_data. register_donor validates and appends; eligibility checks age/last-donation interval and blood group; update_stock modifies a stock record. Prototypes: int register_donor(DonorList*,const Donor*); int eligible(const Donor*,Date); size_t find_donors(const Donor*,size_t,const char*,Donor**,size_t); int update_stock(Stock*,size_t,const char*,int); int save_donors(const char*,const Donor*,size_t); int load_donors(const char*,DonorList*). Use const pointers for read-only inputs and non-const pointers for mutation.

Multiple outputs via pointer parameters: size_t count_eligible(...,int *youngest_id); or return struct Result {size_t count; int youngest_id;}. Structure return groups related outputs and is less error-prone. Globals do not change asymptotic time/space, but increase coupling, complicate tests and prevent independent instances.

A2 – Recursion

Digital root recursion: if n<10 return n; otherwise return digital_root(sum_digits(n)). Power: if exp==0 return 1; half=power(base,exp/2); result=half*half; if exp odd multiply by base. Base cases guarantee termination. Linear power has T(n)=T(n−1)+O(1)=O(n) and n+1 calls; halving has T(n)=T(n/2)+O(1)=O(log n).

Example digital root 9875: 9+8+7+5=29, 2+9=11, 1+1=2. Fast power 2^10 calls exponent sequence 10→5→2→1→0, then unwinds. Missing base case causes unbounded recursion/stack overflow. Iteration is preferable for very deep/simple repetition. Digital root formula for positive n: 1+(n−1)%9; zero maps to zero.

A3 – Pointers and CLI

For int *p=stock: p+3 points to element 3; *(p+3) is its value; p[3] is equivalent; *p++ reads current element then advances p. Memory is contiguous: stock[0]...stock[7]. Function int *lowest_stock(int stock[],int n) returns &stock[min_index], never address of a local variable. CLI examples: bloodbank find O+; bloodbank issue A- 2. Validate argc, group spelling and positive quantity; print usage on invalid input. Returning index instead of pointer is safer after realloc because it can be recomputed; both scans O(n), O(1) space.

A4 – Dynamic memory

Start capacity 2 and double when size==capacity. Use temporary pointer for realloc; on failure retain old allocation and return failure. Pseudocode: newcap=capacity?2*capacity:2; tmp=realloc(data,newcap*sizeof *data); if tmp==NULL return 0; data=tmp; capacity=newcap; append; size++. Leak: forgetting free; dangling pointer: using pointer after free/realloc; double free: freeing same block twice. Set freed pointers to NULL and centralize ownership.

Doubling gives O(n) total copied elements over n appends and O(n) amortized append; fixed +10 growth can copy $O(n^2)$ total. Doubling can leave almost half capacity unused; fixed growth leaves at most 9 slots.

A5 – File storage

Store donor records as text lines with fprintf for human readability and portability; stock fixed-size records can be binary with fwrite/fread for compact fixed offsets. To update stock record i: fseek(fp,i*sizeof(Stock),SEEK_SET); fread; modify units; fseek back by sizeof(Stock) relative to current; fwrite. Check every return value. Text records have variable lengths, so changing one field in place is difficult; rewrite the file. Binary update O(1) record access; rewriting text O(n).

B1 – Instrumented recursion

```
long long power_fast(long long base,int exp){
 calls++;
 if(exp==0)return 1;
 long long half=power_fast(base,exp/2);
 return (exp%2)?half*half*base:half*half;
}
```

Expected for 2^30: linear = 1073741824 in 31 calls; fast = 1073741824 in 6 calls. Calling power_fast twice in the return expression duplicates work because each half is recomputed; call count grows toward a binary recursion tree. Test exponent 0, 1, 10 and 30. Linear recursion calls n+1 times; fast calls floor(log2 n)+1. Numeric overflow of long long occurs for sufficiently large powers and is separate from stack depth.

B2 – Adversarial inputs

scanf("%s") without a width can overflow a name buffer; fgets(buffer,sizeof buffer,stdin) limits input and preserves spaces. Check fopen result and use perror. Validate command-line count and blood-group names. Compile with gcc -Wall -Wextra -fsanitize=address -g. getline reads L characters in O(L) time, allocates heap space as needed, and caller must free the returned buffer.

B3 – Text vs binary

Write/read 10 records and compare every field. For 1k, 10k, 100k records, record file sizes and clock() timings on the same machine; do not invent measured values. Text is portable and inspectable but usually larger and requires parsing. Binary is compact and fast but depends on structure layout, padding, endianness and compiler ABI. Updating record index 3 uses fseek offset 3*sizeof(Stock); index 9 fails because it is beyond EOF and fread returns 0; no write occurs, so file remains unchanged.

```
int update_units(FILE *fp,int index,int units){
 Stock s;
 if(index<0)return 0;
 if(fseek(fp,(long)index*(long)sizeof s,SEEK_SET)!=0)return 0;
 if(fread(&s,sizeof s,1,fp)!=1)return 0;
 s.units=units;
 if(fseek(fp,-(long)sizeof s,SEEK_CUR)!=0)return 0;
 return fwrite(&s,sizeof s,1,fp)==1;
}
```

One fwrite(array,sizeof array element,n,fp) reduces library-call overhead versus n calls, but requires the full array in memory. Time remains O(n); memory O(n).

B4 – Production decision

For 100× donors across 25 hospitals, estimate memory as donor_count×sizeof(Donor)+capacity slack and measure actual peak resident memory. If this fits comfortably, load a growable list for fast O(1) in-memory scans and O(n) eligibility report. If memory is constrained, stream records in batches or maintain a sorted index file for O(log n) disk lookups. Nightly report can scan once O(n), with O(1) extra memory when streaming. Always free allocated memory and check with AddressSanitizer/Valgrind.